

Container Volumes

The Container Lifecycle Problem

A container should be lightweight, stateless, and disposable.

This means:

- Containers start quickly.
- They stop or die at any time.
- They are not meant to store long-term data.
- If a container is removed, **everything inside it disappears.**



Why does a container loose data?

When a developer updates an image or deploys a new version, Docker (or Kubernetes, etc.) behaves like this:

1. Build (or pull) a new image → **old container is destroyed**
2. Create a new container from the updated image → **fresh filesystem**



! The Problem: Databases and Stateful Applications

If your container runs:

- MySQL
- PostgreSQL
- MongoDB
- Redis (when persistence needed)
- Elasticsearch

...then the data is stored on the container's filesystem.

When the container stops → all data is lost.

This is unacceptable for real systems.



The Birth of Volumes - Why They Exist?

The Need for Persistence

We needed a way to:

- Store data **outside** the ephemeral container layer.
- Keep it alive **even if** the container is deleted.
- Allow multiple containers to share common data.
- Allow the host system to manage data safely.

This is where Volumes come into the picture.

A Volume decouples application data from the container lifecycle.



What is Container Volume?

- Stores data **outside** the container's writable layer
- Persists even if the container is recreated, updated, or deleted
- Can be shared across multiple containers
- Is managed by the container engine instead of the user manually handling host paths



Types of Volumes

1. Named Volumes

- Created and managed by Docker
- Docker assigns the actual storage location
- Most commonly used

2. Anonymous Volumes

- Created automatically if no name is provided
- Harder to manage
- Temporary in nature



Types of Volumes

3. Bind Mounts

- Link a host machine directory to the container
- User controls the path
- Useful for development, not ideal for production
- Less isolated and less secure

4. tmpfs Mounts

- Stored in RAM
- Very fast
- Non-persistent
- Used for sensitive or temporary data

